



Security Review For Incentiv Network



Collaborative Audit Prepared For:
Lead Security Expert(s):

Incentiv Network

0xadrii

0xdeadbeef

sammy

Date Audited:

July 28 - August 25, 2025

Introduction

TBA

Scope

Repository: IncentivNetwork/core-contracts

Audited Commit: 731f3c12fb1c9fd199496001b33e6b2866be8229

Final Commit: c2be58205780ff4c693e73cf85aee7325d1c27c1

Files:

- contracts/core/EntryPointSimulations.sol
- contracts/core/EntryPoint.sol
- contracts/incentiv/BaseIncentivAccount.sol
- contracts/incentiv/BulkBalanceReader.sol
- contracts/incentiv/ContractRegistry.sol
- contracts/incentiv/Faucet.sol
- contracts/incentiv/IncentivAccountFactory.sol
- contracts/incentiv/IncentivAccount.sol
- contracts/incentiv/IncentivContractFactory.sol
- contracts/incentiv/RewardDistribution.sol
- contracts/incentiv/TransferGate.sol
- contracts/incentiv/UnifiedTokenPaymaster.sol
- contracts/interfaces/IContractRegistry.sol
- contracts/interfaces/IRewardDistribution.sol
- contracts/interfaces/ITransferGate.sol
- contracts/interfaces/IUniswapV2Router01.sol
- contracts/interfaces/IUniswapV2Router02.sol
- deploy/0_deploy_ContractRegistry.ts
- deploy/1_deploy_EntryPoint.ts
- deploy/2_deploy_IncentivAccountFactory.ts
- deploy/3_deploy_RewardDistribution.ts
- deploy/4_deploy_TransferGate.ts
- deploy/5_deploy_UnifiedTokenPaymaster.ts

- `deploy/6_deploy_IncentivContractFactory.ts`
 - `deploy/7_deploy_BulkBalanceReader.ts`
 - `deploy/8_deploy_Faucet.ts`
 - `hardhat.config.ts`
 - `package.json`
 - `test/entrypointsimulations.test.ts`
 - `test/samples/OracleHelper.test.ts`
 - `test/testutils.ts`
-

Repository: `IncentivNetwork/incentum-pow`

Audited Commit: `0e2b0d065289c9ed19fc398eeb4aa1d176fc62b1`

Final Commit: `afe7ffce52149ca9a50bd17501ef705edb4f9fe0`

Files:

- `allowlist.json`
- `core/txpool/allowlist.go`
- `core/txpool/txpool.go`
- `core/vm/contracts.go`
- `params/protocol_params.go`
- `TESTNET_V2_DEPLOYMENT.md`

Findings

Each issue has an assigned severity:

- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
7	8	8

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: The Paymaster can be drained

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/40>

Summary

The paymaster can be drained by using a malicious contract that doesn't follow Incentiv's account model.

Vulnerability Detail

The `UnifiedTokenPaymaster` contract allows to sponsor gas for users. It's `_validatePaymasterUserOp` works by ensuring that the executions that will be triggered by the caller's `userOp` perform the following actions:

- Call `executeBatch`
- Approve the token to swap to the Swap Router
- Trigger a swap to WETH, and set the receiver of it to be the the Paymaster
- Ensure that the amount being swapped is at least `maxCost`, which is the max cost of the transaction

Essentially, the idea is that users will execute a swap from any token to WETH, where the paymaster is set as destination, effectively paying for the sponsorship using ERC20 tokens.

The problem is that attackers can use a custom contract and be sponsored without ever actually performing the swap. The current logic of the paymaster tries to verify whether the account will perform a swap. However, even if the `calldata` is configured so that a swap would be performed, nothing guarantees that it will actually be performed, because the malicious contract can have custom logic that then skips the swap, so that it is never performed. By doing that, an attacker with a malicious account can simply create a lot of `userOps` and execute them, setting the Incentiv paymaster so that it pays on behalf of them.

Impact

High. The paymaster funds can be drained.

Code Snippet

[UnifiedTokenPaymaster.sol#L58](#)

Tool Used

Manual Review

Recommendation

Consider ensuring that the `userOp` sender is actually an Incentiv account. This can be done by storing the Incentiv account factory, and storing the deployed addresses. Given that users can't upgrade their accounts, this will ensure that the implementation of the `sender` is non-malicious.

Discussion

Birua

Added deployed accounts mapping in `IncentivAccountFactory` and a function to check if an address was deployed by Incentive factory. Check introduced into the Paymaster.
<https://github.com/IncentivNetwork/core-contracts/pull/19/files>

lpetroulakis

Issue fixed in [PR#19](#)

Issue H-2: TransferGate and RewardDistribution should check for transferFrom selector

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/44>

Summary

All token transfer restrictions in the TransferGate can be bypassed by transferring tokens via transferFrom, instead of transfer. In addition, all fees imposed by RewardDistribution can also be bypassed.

Vulnerability Detail

One of the core proposals in Incentiv is to include a TransferGate that allows users to configure whitelists/blacklists. This is done to limit the way in which users can interact between themselves. For example, users can blacklist a certain address to prevent them from transferring tokens to them.

The problem is that the TransferGate only limits this by checking whether the transfer function is being called:

```
// TransferGate.sol
function checkApproval(
    address target,
    uint256 value,
    bytes calldata data
)
    external
    view
    returns (ApprovalResult memory result)
{
    ...SNIP

    // If an ERC20 transfer is being made, check the recipient and sender
    ↪ configurations
    if (data.length == TRANSFER_DATA_LENGTH) {
        ...SNIP
    }
}
```

Malicious users can still use transferFrom to bypass all restrictions imposed by the TransferGate, allowing them to:

- Transfer assets to a globally blacklisted address
- Bypass blacklists/allowlists set by the receiver in accountTargetConfig

Note that this issue is also present in the `RewardDistribution` contract, making it possible to bypass all fees imposed by this contract.

Impact

High, given that the transfer gate is one of Incentiv's main core functionalities and it can be easily bypassed. In addition, all fees imposed by `RewardDistribution` can be bypassed

Code Snippet

- [TransferGate.sol#L188](#).
- [RewardDistribution.sol#L125](#)

Tool Used

Manual Review

Recommendation

Consider also checking for the `transferFrom` selector in both `TransferGate` in `RewardDistribution`. This will prevent fee and restriction bypassing, as it won't be possible to transfer tokens in any way without being affected by the gate and fee requirements.

Discussion

Oxadrii

Fixed in [commit c2c703f](#) and [56a50a3b](#)

Issue H-3: Fees can be bypassed by leveraging the pending transaction mechanism

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/47>

Summary

Users could leverage the early return in pending transactions to avoid paying fees.

Vulnerability Detail

When the receiver has whitelisted the sender in their `IncentivAccount`, all transactions will be set to a pending state:

Note how `_call` will early return if the transaction is set as pending, **and the fee logic is NEVER reached**. Later, this transaction needs to be approved in the `TransferGate`, and the transfer will be directly performed:

```
// File: TransferGate.sol

function approve(
    uint256 pendingId
)
    external
    nonReentrant
{
    ...SNIP

    if (pending.tokenAddress != address(0) && pending.tokenAmount > 0) {
        IERC20(pending.tokenAddress).transfer(pending.tokenRecipient,
            ↪ pending.tokenAmount);
    }

    emit PendingTransactionApproved(pendingId);
}
```

Note how **no fees are applied when the transfer is later accepted in the gate**.

This leads to a scenario where all users can utilize the pending transaction mechanism to bypass all fees.

Impact

High. All fees can be bypassed by leveraging the pending transaction mechanism.

Code Snippet

- [IncentivAccount.sol#L39](#)

Tool Used

Manual Review

Recommendation

Consider applying the corresponding fee when the transaction is later approved in the `TransferGate`, or avoiding early returns and charging the corresponding fee, even if the transaction is set to a `PENDING` state.

Discussion

Birua

Handle fee function added for pending

<https://github.com/IncentivNetwork/core-contracts/pull/24>

lpetroulakis

Issue fixed in [PR#24](#)

Issue H-4: Missing reverting when not having enough balance makes it possible to bypass fees

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/48>

Summary

The incentiv account implementation won't revert if the account doesn't have enough funds, allowing users to always bypass fees.

Vulnerability Detail

Incentiv checks whether the account has enough balance before applying fees, and charges them in case it does:

```
// File: IncentivAccount.sol

function _call(address target, uint256 value, bytes calldata data) internal virtual
↳ override {
    ...SNIP

    // Handle reward distribution
    IRewardDistribution _rdc = rewardDistribution();
    if(address(_rdc) != address(0)) {
        ...SNIP

    // Handle native token fee
    if(feeInfo.fee > 0 && address(this).balance >= value + feeInfo.fee) {
        _rdc.pullFee{value: feeInfo.fee}(target);
    }

    // Handle ERC20 token fee
    if(feeInfo.tokenFee > 0) {
        IERC20 token = IERC20(feeInfo.tokenAddress);
        if(token.balanceOf(address(this)) >= feeInfo.tokenAmount +
↳ feeInfo.tokenFee) { <@
            token.approve(address(_rdc), feeInfo.tokenFee);
            _rdc.pullTokenFee(feeInfo.tokenAddress, feeInfo.tokenRecipient,
↳ feeInfo.tokenFee);
        }
    }
}
}
```

The problem is that there's no revert if the account doesn't have enough funds to pay for

the fee, allowing accounts to always avoid fees by performing transactions that leave the account without enough balance to pay for the fee.

Impact

High. Fees can be bypassed due to not reverting when there's not enough funds

Code Snippet

- [IncentivAccount.sol#L58](#)
- [IncentivAccount.sol#L65](#)

Tool Used

Manual Review

Recommendation

Revert if the account doesn't have enough funds to pay for the corresponding fee.

Discussion

lpetroulakis

Issue fixed in [PR#25](#)

Issue H-5: Webauthn precompile input validation DOS

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/59>

Summary

The WebAuthn precompile lacks proper input length validation, allowing attackers to crash nodes by providing maliciously crafted inputs that cause index out-of-bounds panics.

Vulnerability Detail

The WebAuthn precompile function directly accesses input bytes without validating the input length first:

```
func (c *webAuthnVerify) Run(input []byte) ([]byte, error) {
    challenge := input[:32]
    authDataLen := binary.BigEndian.Uint32(input[32:36])

    authenticatorData := input[36 : 36+authDataLen]
    requireUserVerification := input[36+authDataLen] == 1

    offset := 37 + authDataLen
    clientDataJSONLen := binary.BigEndian.Uint32(input[offset : offset+4])

    clientDataJSON := string(input[offset+4 : offset+4+clientDataJSONLen])

    // More unsafe slice operations...
}
```

Critical Issues:

1. **No Minimum Length Check:** The function immediately tries to access `input[:32]` and `input[32:36]` without checking if `len(input) >= 36`
2. **Unbounded Memory Allocation:** `authDataLen` and `clientDataJSONLen` are derived from untrusted input and can be set to arbitrary large values (up to `uint32` max = 4GB), causing:
 - Excessive memory allocation in `string(input[offset+4 : offset+4+clientDataJSONLen])`
 - Out-of-memory crashes
 - DoS through resource exhaustion

3. **Index Out-of-Bounds:** Multiple slice operations can panic if the calculated offsets exceed the input length:
 - `input[36 : 36+authDataLen]`
 - `input[offset+4 : offset+4+clientDataJSONLen]`
 - `input[offset+8 : offset+40]` (signature components)
4. **Integer Overflow:** Calculations like `offset = 37 + authDataLen` and `offset+4+clientDataJSONLen` can overflow, wrapping around to small values and causing out-of-bounds access

Impact

- **Node Crashes:** Index out-of-bounds panics can crash nodes, causing network disruption
- **DoS Attacks:** Attackers can cheaply crash multiple nodes with a single malicious transaction
- **Resource Exhaustion:** Large memory allocations can exhaust node memory and crash the process
- **Network Instability:** Widespread node crashes can destabilize the entire network
- **Service Unavailability:** Critical infrastructure nodes going down can make the network unusable

Code Snippet

```
// Vulnerable code in incentum-pow/core/vm/contracts.go:1116-1136
func (c *webAuthnVerify) Run(input []byte) ([]byte, error) {
    challenge := input[:32] // @audit - Panic if len(input) < 32
    authDataLen := binary.BigEndian.Uint32(input[32:36]) // @audit - Panic if
    ↪ len(input) < 36

    authenticatorData := input[36 : 36+authDataLen] // @audit - Panic if len(input)
    ↪ < 36+authDataLen
    requireUserVerification := input[36+authDataLen] == 1 // @audit - Panic if
    ↪ len(input) <= 36+authDataLen

    offset := 37 + authDataLen // @audit - Can overflow
    clientDataJSONLen := binary.BigEndian.Uint32(input[offset : offset+4]) //
    ↪ @audit - Panic if out of bounds

    clientDataJSON := string(input[offset+4 : offset+4+clientDataJSONLen]) //
    ↪ @audit - Massive memory allocation

    // More unsafe operations...
```

```
r := new(big.Int).SetBytes(input[offset+8 : offset+40]) // @audit - Panic if
↳ out of bounds
s := new(big.Int).SetBytes(input[offset+40 : offset+72]) // @audit - Panic if
↳ out of bounds
x := new(big.Int).SetBytes(input[offset+72 : offset+104]) // @audit - Panic if
↳ out of bounds
y := new(big.Int).SetBytes(input[offset+104 : offset+136]) // @audit - Panic if
↳ out of bounds
}
```

Tool Used

Manual Review

Recommendation

1. Check minimum input length before any slice operations
2. Set reasonable upper bounds on variable-length fields
3. Use safe arithmetic to prevent integer overflow
4. Validate all calculated offsets before use
5. Follow the pattern used by other precompiles in geth

Discussion

lpetroulakis

Issue fixed in [PR#28](#)

Issue H-6: setRecoveryAddressWithPermit is replayable

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/61>

Summary

BaseIncentivAccount.sol's setRecoveryAddressWithPermit is replayable. Once a valid signature is used to set a recovery address, the same signature can be reused by anyone to re-set that same address indefinitely. The signature only covers keccak256(newRecoveryAddress) with no nonce/deadline/domain binding.

Vulnerability Detail

```
function setRecoveryAddressWithPermit(
    address newRecoveryAddress,
    bytes calldata signature
)
public
{
    bytes32 message = keccak256(abi.encodePacked(newRecoveryAddress));
    require(
        _internalSignatureValidation(message, signature),
        "Invalid signature"
    );
    recoveryAddress = newRecoveryAddress;
}
```

As can be seen above - `recoveryAddress = newRecoveryAddress;` is set as long as `_internalSignatureValidation` returns true, which it does with a valid signature.

There is no nonce or signature deny list that prevents this signature from being used again.

Impact

Malicious actors can set the recover address to a recovery address that was already set in the past. In-cases where that address is not in control of the owner, the account can get locked.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider using a EIP712 domain with proper nonce/deadline/domain binding.

Discussion

lpetroulakis

Issue fixed in [PR#23](#)

Issue H-7: User can upgrade the the account proxy implementation

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/62>

Summary

The account proxy implementation is in control of the user. This is because `_authorizeUpgrade` access is controlled through `_onlyOwner` where owner is controlled - the user can change the implementation. This means the user can for example bypass the `TransferGate` and fees and any other logic of the account.

Vulnerability Detail

To authorize an upgrade - `_onlyOwner()` must succeed

```
function _authorizeUpgrade(address newImplementation) internal view override {
    (newImplementation);
    _onlyOwner();
}
```

`_onlyOwner()` allows three cases for authority:

1. caller is recovery address
2. caller is owner (EOA)
3. caller is the account it self (self call)

```
function _onlyOwner() internal view {
    // directly from EOA/Passkey owner, through recovery address or through the
    // account itself (which gets redirected through execute())
    require(
        msg.sender == address(this) ||
        (msg.sender == owner && owner != address(0)) ||
        (msg.sender == recoveryAddress && recoveryAddress != address(0)),
        "only owner"
    );
}
```

In any of the above imitated calls, the user can change the account implementation.

Impact

Total control over the implementation. Impact example - user can bypass the `TransferGate` and fees and any other logic of the account.

Code Snippet

Tool Used

Manual Review

Recommendation

Use a beacon proxy pattern.

Discussion

Birua

Beacon proxy pattern is implemented: <https://github.com/IncentivNetwork/core-contracts/commit/e38a38860682b2d906f6f3a5ed73a91ba536911c>

lpetroulakis

Issue fixed in commit [e38a38860682b2d906f6f3a5ed73a91ba536911c](https://github.com/IncentivNetwork/core-contracts/commit/e38a38860682b2d906f6f3a5ed73a91ba536911c)

Issue M-1: Not accounting for postOp gas cost when refunding ETH favors the refundee

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/41>

Summary

The `_postOp` refund logic does not consider the `postOp` gas cost, always refunding more to the `userOp` sender.

Vulnerability Detail

In `_postOp`, the excess amount of gas paid by the user will be refunded to them:

```
// File: UnifiedTokenPaymaster.sol

function _postOp(
    PostOpMode /*mode*/,
    bytes calldata context,
    uint256 actualGasCost,
    uint256 /*actualUserOpFeePerGas*/
) internal virtual override {
    // Decode the context data
    (address sender, uint256 amountOut, uint256 preVerificationGas) =
        ↪ abi.decode(context, (address, uint256, uint256));
    if (amountOut > actualGasCost + preVerificationGas + GAS_REFUND) {
        // Send excess amount to sender
        (bool success, ) = sender.call{value: amountOut - actualGasCost -
            ↪ preVerificationGas - GAS_REFUND, gas: GAS_REFUND}("");
        // We dont want to fail if not refunded successfully, so supress warning
        success;
    }
}
```

Note how the refunded gas is derived from `amountOut - actualGasCost - preVerificationGas - GAS_REFUND`, where:

- `amountOut` is the output of the Swap
- `actualGasCost` is the cost of the `userOp` execution + the `preOpGas`
- `preVerificationGas` is the `preVerificationGas` value specified by the user in the `userOp`
- `GAS_REFUND` is the cost of sending ETH back to the user in `postOp`

Given that `actualGasCost` does not consider the postOp gas cost, more gas than expected will always be refunded to the user, given that the refund logic does not account for the cost and assets spent on calling `postOp`.

Impact

Medium. More funds than expected will be refunded to the user. On the long term, this will lead to a small drain of the paymaster.

Code Snippet

[UnifiedTokenPaymaster.sol#L135](#)

Tool Used

Manual Review

Recommendation

Consider accounting for the `postOp` gas in the gas refund. This can be done by setting a configured value in the Paymaster with the proper estimation of the gas cost of `postOp`.

Discussion

lpetroulakis

Issue fixed in [PR#20](#)

Issue M-2: Incorrect addition of GAS_REFUND in _postOp leads to wrong refund computation

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/42>

Summary

In `_postOp`, `GAS_REFUND` is used to compute the total cost of the gas used. However, `GAS_REFUND` is a gas amount, not a gas value, leading to an incorrect computation.

Vulnerability Detail

`_postOp` will compute the excess amount of gas paid by the user and refund it to them. The total gas cost consumed is obtained from `actualGasCost + preVerificationGas + GAS_REFUND`:

```
// File: UnifiedTokenPaymaster.sol

function _postOp(
    PostOpMode /*mode*/,
    bytes calldata context,
    uint256 actualGasCost,
    uint256 /*actualUserOpFeePerGas*/
) internal virtual override {
    // Decode the context data
    (address sender, uint256 amountOut, uint256 preVerificationGas) =
        ↪ abi.decode(context, (address, uint256, uint256));
    if (amountOut > actualGasCost + preVerificationGas + GAS_REFUND) {
        // Send excess amount to sender
        (bool success, ) = sender.call{value: amountOut - actualGasCost -
            ↪ preVerificationGas - GAS_REFUND, gas: GAS_REFUND}("");
        // We dont want to fail if not refunded successfully, so supress warning
        success;
    }
}
```

The problem is that in the `actualGasCost + preVerificationGas + GAS_REFUND`, `GAS_REFUND` isn't actually a cost, but a gas amount. This will lead to an incorrect calculation of the gas cost for refunding the ETH, leading to higher refunds than expected.

Impact

Medium. The incorrect use of `GAS_REFUND` instead of gas refund cost will lead to always refunding more assets than expected.

Code Snippet

[UnifiedTokenPaymaster.sol#L135](#)

Tool Used

Manual Review

Recommendation

Consider updating the calculation so that the cost of refunding the gas is used, instead of the `GAS_REFUND`, which is the amount of gas. The `maxFeePerGas` should be encoded in the paymaster context so that such computation can be performed.

Discussion

Birua

`maxFeePerGas` is now encoded into the context and `postOp` calculation is corrected. `_validatePaymasterUserOp` had to be restructured because of stack too deep error.
<https://github.com/IncentivNetwork/core-contracts/pull/20/files>

Ipetroulakis

Issue fixed in [PR#20](#)

Issue M-3: The fee cap mechanism can't be used

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/46>

Summary

feeCap is used as a global value, instead of on a per-token basis. This makes it impossible to properly configure fee caps.

Vulnerability Detail

The RewardDistribution contract's getFee function incorporates a fee limitation mechanism by using a storage value feeCap:

```
// File: RewardDistribution.sol

function getFee(
    address target,
    uint256 value,
    bytes calldata data,
    uint256 /*gasUsed*/
)
    external
    view
    returns (FeeInfo memory feeInfo)
{
    // Calculate native token fee if value > 0
    if (value > 0) {
        uint256 fee = (value * feePercentage) / 100000;
        feeInfo.fee = fee > feeCap ? feeCap : fee;
    }

    // Try to parse token transfer
    if (data.length == TRANSFER_DATA_LENGTH) {
        (bool isValid, address recipient, uint256 amount) = parseTransfer(data);
        if (isValid && amount > 0) {
            uint256 tokenFee = (amount * feePercentage) / 100000;
            feeInfo.tokenFee = tokenFee > feeCap ? feeCap : tokenFee;
            feeInfo.tokenAmount = amount;
            feeInfo.tokenAddress = target;
            feeInfo.tokenRecipient = recipient;
        }
    }

    return feeInfo;
}
```


As shown in the snippet, a `feePercentage` is applied to the transferred `value/amount`. If the resulting fee exceeds the `feeCap`, only `feeCap` is charged.

The problem is that the `feeCap` doesn't limit the fee applied as a max percentage. Instead, it is checked as a global value.

For example, if 1 ETH is transferred and a 10% fee is applied, `fee` will be 0.1 ETH. This will then be directly compared with `feeCap`, which forces `feeCap` to be denominated in ETH for this case (let's say it's configured to `0.2e18`). Because it is a global value, if we then perform a USDC transfer (6 decimals), there's no way to limit the fee applied, given that `feeCap` is stored as an 18-decimal value.

Impact

Medium. The `feeCap` mechanism can't be effectively used, making it impossible to limit fees.

Code Snippet

- [RewardDistribution.sol#L121](#)

Tool Used

Manual Review

Recommendation

Consider making `feeCap` limit fees applied as a max percentage of the total value, or configure it on a per-token basis, instead of a global value.

Discussion

lpetroulakis

Issue fixed in [PR#26](#)

Issue M-4: userOps with `initCode.length != 0` can be DoS'ed

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/51>

Summary

Anybody can call `createSender`, making it possible to frontrun userOps and DoS them.

Vulnerability Detail

When a `userOp` is triggered, it is possible to specify an `initCode.length != 0` to signal that the account should be created via the `SenderCreator`'s `createSender` function:

```
// File: EntryPoint.sol
function _createSenderIfNeeded(
    uint256 opIndex,
    UserOpInfo memory opInfo,
    bytes calldata initCode
) internal {
    if (initCode.length != 0) {
        address sender = opInfo.mUserOp.sender;
        if (sender.code.length != 0)
            revert FailedOp(opIndex, "AA10 sender already constructed");
        address sender1 = senderCreator().createSender{
            gas: opInfo.mUserOp.verificationGasLimit
        }(initCode);
        ...SNIP
    }
}
```

The problem is that `CreateSender`'s `createSender` function is permissionless. Because of this, the following attack can be performed:

1. A user signs and submits a `userOp` with `initCode.length != 0`, specifying that the account should be deployed.
2. A malicious user frontruns the transaction and calls `createSender`, making `sender.code.length > 0`.
3. When the `_createSenderIfNeeded` function is called in the `EntryPoint`, `initCode.length` will be `> 0` (as the `userOp` can't be changed), but then `sender.code.length != 0` as the attacker will have already created the account. This will lead to the `userOp` transaction reverting with "AA10 sender already constructed".

Impact

Medium. All userOp's where the account should be freshly deployed can be effectively DoS'ed.

Code Snippet

- [SenderCreator.sol#L17](#)

Tool Used

Manual Review

Recommendation

Consider only allowing `createSender` in the `SenderCreator` contract to be callable by the `EntryPoint`, just like in the [updated EntryPoint 0.8 version](#).

Discussion

Oxadrii

Fixed in [commit b1f622b6](#)

Issue M-5: Inappropriate log level usage

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/52>

Summary

The allowlist module uses `log.Info` for debug-level information that should use `log.Debug` instead, potentially flooding production logs and causing the node to run out of storage.

Vulnerability Detail

On line 97 in the `IsSenderAllowed()` function, there is a `log.Info` call that logs the result of every sender allowlist check:

```
log.Info("allowlist: sender check result", "address", addr.Hex(), "allowed",  
↪ allowed)
```

This logging statement will be triggered for every transaction validation, which can happen thousands of times per second in a busy network. Using `log.Info` for this type of detailed operational information is inappropriate because:

1. Info-level logs are typically enabled in production
2. This creates excessive log volume for routine operations
3. The information is more suited for debugging than general information

Impact

- **Log flooding:** Production logs will be overwhelmed with allowlist check results
- **Performance degradation:** Excessive logging can impact node performance
- **Storage costs:** Large log files consume unnecessary disk space

Code Snippet

```
func IsSenderAllowed(addr common.Address) bool {  
    mu.RLock()  
    defer mu.RUnlock()  
  
    // ... validation logic ...  
  
    _, allowed := allowedSenders[addr]  
    log.Info("allowlist: sender check result", "address", addr.Hex(), "allowed",  
    ↪ allowed) // @audit avoid using log.Info
```

```
    return allowed
}
```

Tool Used

Manual Review

Recommendation

Change the log level from `log.Info` to `log.Debug` or remove the logging entirely:

```
func IsSenderAllowed(addr common.Address) bool {
    mu.RLock()
    defer mu.RUnlock()

    // ... validation logic ...

    _, allowed := allowedSenders[addr]
    log.Debug("allowlist: sender check result", "address", addr.Hex(), "allowed",
        ↪ allowed)
    return allowed
}
```

Discussion

lpetroulakis

Issue fixed in [PR#11](#)

Issue M-6: Underpriced precompile gas costs

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/55>

Summary

The P256VerifyGas and WebAuthnVerifyGas constants are significantly underpriced compared to the official Ethereum specification, creating potential DoS vectors and resource exhaustion attacks.

Vulnerability Detail

The current implementation sets both P256 and WebAuthn verification gas costs at 3000 gas units:

```
// P256 precompiled contract gas prices
P256VerifyGas uint64 = 3000 // Gas price for P256 signature verification

// Gas costs for WebAuthn verification
WebAuthnVerifyGas uint64 = 3000 // Gas costs for WebAuthn verification //@audit
↪ seems underpriced
```

However, according to EIP-7951 (<https://eips.ethereum.org/EIPS/eip-7951>), the official Ethereum specification has updated the P256VerifyGas to 6900 gas units as of the recent implementation in the official geth repository (https://github.com/ethereum/go-ethereum/blob/792de5d2e3b40837aadb65993e387bf5aa5bce9f/params/protocol_params.go#L168).

The significant underpricing creates several issues:

1. **P256 verification is computationally expensive** but only costs 3000 gas vs the recommended 6900 gas
2. **WebAuthn verification is even more expensive** as it performs additional operations (JSON parsing, string operations, SHA-256 hashing, authenticator data validation) on top of P256 verification, yet costs the same 3000 gas
3. **Gas cost doesn't reflect computational complexity**, making these precompiles attractive targets for resource exhaustion attacks

Impact

- **DoS attacks:** Attackers can cheaply consume excessive computational resources by calling underpriced precompiles
- **Economic imbalance:** Users get subsidized expensive cryptographic operations at the expense of network stability

- **Consensus risk:** Potential for block processing delays or validation failures under heavy precompile usage

Code Snippet

```
// lines 159-162 in incentum-pow/params/protocol_params.go
// P256 precompiled contract gas prices
P256VerifyGas uint64 = 3000 // Gas price for P256 signature verification

// Gas costs for WebAuthn verification
WebAuthnVerifyGas uint64 = 3000 // Gas costs for WebAuthn verification //@audit
↪ seems underpriced
```

Tool Used

Manual Review

Recommendation

Update the gas costs to reflect the computational complexity and align with official Ethereum specifications.

References

- Official EIP-7951: <https://eips.ethereum.org/EIPS/eip-7951>
- Updated geth implementation:
https://github.com/ethereum/go-ethereum/blob/792de5d2e3b40837aadb65993e387bf5aa5bce9f/params/protocol_params.go#L168

Discussion

lpetroulakis

Issue fixed in [PR#30](#)

Issue M-7: Node is affected by some go-ethereum vulnerabilities

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/56>

Summary

Using 1.11 version of geth makes Incentiv vulnerable to some vulnerabilities found in that version.

Vulnerability Detail

There's some security issues that affect the 1.11 version of geth. Specifically, the node is affected by:

- <https://github.com/ethereum/go-ethereum/security/advisories/GHSA-4xc9-8hmq-j652>
- <https://github.com/ethereum/go-ethereum/security/advisories/GHSA-ppjg-v974-84cm>

Both of them are "Dos via malicious p2p messages" type of issues, which allow the node to be effectively DoS'ed.

Impact

Medium.

Code Snippet

- [accessors_chain.go#L337](#)
- [peer.go#L295](#)

Tool Used

Manual Review

Recommendation

Consider applying the following fixes, extracted from the official Geth Security Advisory:

- [PR 29534](#)
- [PR 27887](#)

Discussion

lpetroulakis

Issue fixed in [PR#21](#)

Issue M-8: Light client txpool missing allowlist validation

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/57>

Summary

The light client transaction pool lacks allowlist validation checks that are present in the main client, allowing attackers to bypass transaction filtering and potentially DoS the network through invalid transaction gossip.

Vulnerability Detail

The light client's TxPool implementation in `incentum-pow/light/txpool.go` performs basic transaction validation in the `validateTx` function but does not include allowlist checks that are implemented in the main transaction pool (`incentum-pow/core/txpool/txpool.go`).

The main client includes allowlist validation through:

- `IsSenderAllowed()` - validates transaction senders against an allowlist
- `IsReceiverAllowed()` - validates transaction receivers against an allowlist

This discrepancy allows attackers to:

1. **Submit transactions through light clients** that would be rejected by full nodes due to allowlist restrictions
2. **Bypass allowlist controls** by routing transactions through light clients instead of full nodes
3. **Flood the gossip network** with invalid transactions that consume bandwidth and processing resources

Impact

- **DoS attacks on light clients:** Attackers can overwhelm light clients with invalid transactions that bypass allowlist checks
- **Network resource waste:** Invalid transactions get gossiped to full nodes, consuming bandwidth and processing resources
- **Gossip protocol stress:** High volume of invalid transactions creates unnecessary load on the peer-to-peer network

Code Snippet

```
// Light client validation (incentum-pow/light/txpool.go:349-396)
func (pool *TxPool) validateTx(ctx context.Context, tx *types.Transaction) error {
    // Validate sender signature
    if from, err = types.Sender(pool.signer, tx); err != nil {
        return txpool.ErrInvalidSender
    }
    // ... other validations (nonce, gas, balance, intrinsic gas)
    // MISSING: Allowlist validation checks
    return currentState.Error()
}

// Missing allowlist checks that exist in main client:
// - IsSenderAllowed(from)
// - IsReceiverAllowed(tx.To())
```

Tool Used

Manual Review

Recommendation

Integrate allowlist validation into the light client transaction pool by importing and using the same allowlist functionality:

```
import (
    "github.com/ethereum/go-ethereum/core/txpool"
    // Add allowlist import - adjust path as needed
)

func (pool *TxPool) validateTx(ctx context.Context, tx *types.Transaction) error {
    // Existing validations...
    var (
        from common.Address
        err error
    )

    if from, err = types.Sender(pool.signer, tx); err != nil {
        return txpool.ErrInvalidSender
    }

    // Add allowlist validation
    if !txpool.IsSenderAllowed(from) {
        return txpool.ErrSenderNotAllowed
    }
}
```

```
if tx.To() != nil && !txpool.IsReceiverAllowed(tx.To()) {  
    return txpool.ErrReceiverNotAllowed  
}  
  
// Continue with existing validations...  
currentState := pool.currentState(ctx)  
if n := currentState.GetNonce(from); n > tx.Nonce() {  
    return core.ErrNonceTooLow  
}  
  
// ... rest of existing validation logic  
}
```

Discussion

lpetroulakis

Issue fixed in [PR#32](#)

Issue L-1: SetAllowlistFilePath can never be called

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/39>

Summary

The SetAllowlistFilePath in allowlist.go is uncallable.

Vulnerability Detail

SetAllowlistFilePath is defined in allowlist.go but never exposed outside of it, making the function effectively uncallable.

Impact

Informational.

Code Snippet

[allowlist.go#L72](#)

Tool used

Manual review.

Recommendation

Consider making the function callable by exposing it via RPC.

Discussion

Oxadrii

Acknowledged, the function is used for testing purposes

crypto-zk

We removed this function in this PR (merged):

<https://github.com/IncentivNetwork/incentum-pow/pull/17/files>

crypto-zk

However, since it is used in many tests in core/txpool/allowlist_test.go, we are returning it in PR <https://github.com/IncentivNetwork/incentum-pow/pull/32/files>.

crypto-zk

If the function is only needed for testing, it can probably be removed. However, the tests will then not work. What do you think?

crypto-zk

I think that within the scope of [PR #32](#), we should not touch `core/txpool/allowlist.go` at all, so I am removing this function from this PR. If we decide that `SetAllowlistFilePath` is still needed, then we should probably revert [PR #17](#).

crypto-zk

In addition, in the new implementation of `SetAllowlistFilePath` in [PR #32](#), `loadAllowlistFromFile()` is missing at the end of the function, which means that the list is not updated immediately. The update (`checkAndReloadAllowlist()`) will only occur when (within 5 seconds) the background process (goroutine) launched in the `init()` function is triggered.

Oxadrii

The fix looks good, agree as the function wasn't calling `loadAllowlistFromFile()` so would have not been 100% effective either. There shouldn't be any issue with the function being there as it wasn't exposed but imo it's not needed and it can be safely removed. For the rest it looks good

lpetroulakis

Issue fixed in [PR#17](#)

Issue L-2: Adding preVerificationGas in the postOp gas computation is redundant

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/43>

Summary

Adding preVerificationGas in the postOp gas computation is redundant, given that preVerificationGas is encoded as 0 in paymaster userOp validation.

Vulnerability Detail

In Paymaster's _validatePaymasterUserOp, a 0 is encoded as the preVerificationGas:

```
// File: UnifiedTokenPaymaster.sol

function _validatePaymasterUserOp(
    PackedUserOperation calldata userOp,
    bytes32 /*userOpHash*/,
    uint256 maxCost
) internal virtual override returns (bytes memory context, uint256
    ↪ validationData) {

    ...SNIP

    // Pass sender and amountOut
    return (abi.encode(sender, amountOut, 0), 0); <@ Third param is
    ↪ `preVerificationGas`

}
```

Then, in _postOp, this value is used to compute the gas cost utilized so far:

```
// File: UnifiedTokenPaymaster.sol

function _validatePaymasterUserOp(
function _postOp(
    PostOpMode /*mode*/,
    bytes calldata context,
    uint256 actualGasCost,
    uint256 /*actualUserOpFeePerGas*/
) internal virtual override {
    // Decode the context data
    (address sender, uint256 amountOut, uint256 preVerificationGas) =
    ↪ abi.decode(context, (address, uint256, uint256));
```

```

        if (amountOut > actualGasCost + preVerificationGas + GAS_REFUND) { <@
            ...SNIP
        }
    }
}

```

Given that `actualGasCost` already includes the pre-verification cost, it is correct to encode it as 0 in the paymaster validation. However, it is then not necessary to include it in the refund computation, as it will always be zero.

Impact

Gas/Informational.

Code Snippet

[UnifiedTokenPaymaster.sol#L135](#)

Tool Used

Manual Review

Recommendation

Consider removing `preVerificationGas` from the refund computation:

```

function _postOp(
    PostOpMode /*mode*/,
    bytes calldata context,
    uint256 actualGasCost,
    uint256 /*actualUserOpFeePerGas*/
) internal virtual override {
    // Decode the context data
    - (address sender, uint256 amountOut, uint256 preVerificationGas) =
    ↪ abi.decode(context, (address, uint256, uint256));
    + (address sender, uint256 amountOut, ) = abi.decode(context, (address,
    ↪ uint256, uint256));
    - if (amountOut > actualGasCost + preVerificationGas + GAS_REFUND) {
    + if (amountOut > actualGasCost + GAS_REFUND) {
    - (bool success, ) = sender.call{value: amountOut - actualGasCost -
    ↪ preVerificationGas - GAS_REFUND, gas: GAS_REFUND}("");
    + (bool success, ) = sender.call{value: amountOut - actualGasCost - GAS_REFUND,
    ↪ gas: GAS_REFUND}("");
        // We dont want to fail if not refunded successfully, so supress warning
        success;
    }
}

```



```
}
```

Discussion

lpetroulakis

Issue fixed in [PR#20](#)

Issue L-3: `safeTransfer/safeTransferFrom` should be used across all contracts

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/45>

Summary

For all ERC20 token transfers, `safeTransfer/safeTransferFrom` should be used to avoid issues with weird ERC20 token implementations.

Vulnerability Detail

ERC20 transfers performed in all contracts miss using safety transfer libraries. This can lead to some issues. For example, in the `RewardDistribution` contract, USDT users can avoid paying fees, given that if user doesn't have enough funds, the fee transfer will still not revert (as USDT returns a boolean instead of reverting).

Impact

Low.

Code Snippet

- [RewardDistribution.sol#L96](#)
- [TransferGate.sol#L357](#)

Tool Used

Manual Review

Recommendation

Consider using safe ERC20 transfer libraries to perform all token transfers.

Discussion

Oxadrii

Fixed in commits [b07430f5](#) and [a49de992](#).

Issue L-4: IncentivAccount checks balance for fee payment incorrectly

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/49>

Summary

In fee handling, the account balance is required to be $\geq \text{value transferred} + \text{fee}$, instead of $\geq \text{fee}$, incorrectly requiring more funds than necessary to be held in the account.

Vulnerability Detail

When fees are applied in Incentiv, a check is performed to ensure that the account has enough balance to pay for the fee:

```
// File: IncentivAccount.sol

function _call(address target, uint256 value, bytes calldata data) internal virtual
↳ override {
    ...SNIP

    // Handle reward distribution
    IRewardDistribution _rdc = rewardDistribution();
    if(address(_rdc) != address(0)) {
        ...SNIP

    // Handle native token fee
    if(feeInfo.fee > 0 && address(this).balance >= value + feeInfo.fee) {
        _rdc.pullFee{value: feeInfo.fee}(target);
    }

    // Handle ERC20 token fee
    if(feeInfo.tokenFee > 0) {
        IERC20 token = IERC20(feeInfo.tokenAddress);
        if(token.balanceOf(address(this)) >= feeInfo.tokenAmount +
↳ feeInfo.tokenFee) { <@
            token.approve(address(_rdc), feeInfo.tokenFee);
            _rdc.pullTokenFee(feeInfo.tokenAddress, feeInfo.tokenRecipient,
↳ feeInfo.tokenFee);
        }
    }
}
}
```

As shown, the balance checks are incorrect:

- For ETH fees, `address(this).balance` is incorrectly required to be `>=` than `value + feeInfo.fee`, instead of only the `feeInfo.fee` applied
- For token transfers, `token.balanceOf(address(this))` is incorrectly required to be `>=` `feeInfo.tokenAmount + feeInfo.tokenFee`, instead of only the `feeInfo.tokenFee` applied.

Impact

Informational. More funds than necessary are required to be held in the account so that the fee charging logic can be triggered. In situations where the account could pay for the fee, the fee won't actually be charged.

Code Snippet

Tool Used

Manual Review

Recommendation

Consider only checking whether balance is enough to cover the fee:

```
function _call(address target, uint256 value, bytes calldata data) internal virtual
↳ override {

    ..SNIP

    IRewardDistribution _rdc = rewardDistribution();
    if(address(_rdc) != address(0)) {
        IRewardDistribution.FeeInfo memory feeInfo = _rdc.getFee(target, value,
↳ data, preCallGas - postCallGas);

        // Handle native token fee
-       if(feeInfo.fee > 0 && address(this).balance >= value + feeInfo.fee) {
+       if(feeInfo.fee > 0 && address(this).balance >= feeInfo.fee) {
            _rdc.pullFee{value: feeInfo.fee}(target);
        }

        // Handle ERC20 token fee
        if(feeInfo.tokenFee > 0) {
            IERC20 token = IERC20(feeInfo.tokenAddress);
-            if(token.balanceOf(address(this)) >= feeInfo.tokenAmount +
↳ feeInfo.tokenFee) {
```

```

+         if(token.balanceOf(address(this)) >= feeInfo.tokenFee) {
            token.approve(address(_rdc), feeInfo.tokenFee);
            _rdc.pullTokenFee(feeInfo.tokenAddress, feeInfo.tokenRecipient,
                ↪ feeInfo.tokenFee);
        }
    }
}

```

Discussion

Oxadrii

Partially fixed in [commit 39d17974](#), misses adjusting native fee check

Birua

Native fee check corrected

<https://github.com/IncentivNetwork/core-contracts/pull/21>

lpetroulakis

Issue fixed in [PR#21](#)

Issue L-5: Updates in ContractRegistry to change the EntryPoint address won't be reflected in accounts

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/50>

Summary

Updates in ContractRegistry to update the Entry Point won't be reflected in accounts, forcing account redeployment.

Vulnerability Detail

Incentiv accounts store the ContractRegistry, which stores all of the Incentiv platform contracts.

In order to fetch the most updated contracts, the account interacts with the registry:

```
// File: IncentivAccount.sol

function rewardDistribution() public view returns (IRewardDistribution) {
    return IRewardDistribution(contractRegistry().getContract(IContractRegistry
    ↪ .IncentivContracts.REWARD_DISTRIBUTION));
}
```

However, this is not done for the EntryPoint:

```
// File: IncentivAccount.sol

function _requireFromEntryPointOrOwner() internal view {
    require(
        msg.sender == address(entryPoint()) ||
        (msg.sender == owner && owner != address(0)) ||
        (msg.sender == recoveryAddress && recoveryAddress != address(0)),
        "account: not Owner or EntryPoint"
    );
}

function entryPoint() public view virtual override returns (IEntryPoint) {
    return _entryPoint; <@ Returns stored _entryPoint, instead of interacting
    ↪ with registry
}
```

This can lead to situations where if the EntryPoint is upgraded in the registry (for example upgrading to version 0.8 of the EntryPoint), it won't be reflected in the Incentiv Accounts,

forcing ALL of them to be upgraded.

Impact

Low.

Code Snippet

- [BaseIncentivAccount.sol#L54](#)

Tool Used

Manual Review

Recommendation

Consider fetching the EntryPoint from the registry:

```
// File: BaseIncentivAccount.sol

function entryPoint() public view virtual override returns (IEntryPoint) {
-     return _entryPoint;
+     return IEntryPoint(contractRegistry().getContract(IContractRegistry.Incent
↪   ivContracts.ENTRY_POINT));
}
```

Discussion

Birua

BaseIncentivAccount get entryPoint from contractRegistry. Required constructor changes because entryPoint address is no longer needed as an argument.

<https://github.com/IncentivNetwork/core-contracts/pull/22/files>

lpetroulakis

Issue fixed in [PR#22](#)

Issue L-6: Redundant empty list checks

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/53>

Summary

The allowlist validation functions contain redundant checks for empty lists that result in unnecessary computation and code duplication.

Vulnerability Detail

In both `IsSenderAllowed()` and `IsReceiverAllowed()` functions, there are redundant checks for empty allowlists:

1. Lines 86-88: Check if both sender and receiver lists are empty
2. Lines 91-93 (sender) / 116-118 (receiver): Check if the specific list is empty

The second check is redundant because if the specific list is empty, the first condition would have already returned true (assuming the other list is also empty, or if not, the specific empty list should allow all).

Code Snippet

```
// IsSenderAllowed - lines 86-98
if len(allowedSenders) == 0 && len(allowedReceivers) == 0 { // @audit redundant
    return true
}

// If only sender list is empty, allow all senders
if len(allowedSenders) == 0 {
    return true
}

// IsReceiverAllowed - lines 111-118
if len(allowedSenders) == 0 && len(allowedReceivers) == 0 { // @audit redundant
    return true
}

// If only receiver list is empty, allow all receivers
if len(allowedReceivers) == 0 {
    return true
}
```


Tool Used

Manual Review

Recommendation

Simplify the logic by removing the redundant checks:

```
func IsSenderAllowed(addr common.Address) bool {
    mu.RLock()
    defer mu.RUnlock()

    // If sender list is empty, allow all senders
    if len(allowedSenders) == 0 {
        return true
    }

    // Check if the address is in the sender list
    _, allowed := allowedSenders[addr]
    return allowed
}

func IsReceiverAllowed(addr *common.Address) bool {
    mu.RLock()
    defer mu.RUnlock()

    if addr == nil { // contract creation always allowed
        return true
    }

    // If receiver list is empty, allow all receivers
    if len(allowedReceivers) == 0 {
        return true
    }

    // Check if the address is in the receiver list
    _, allowed := allowedReceivers[*addr]
    return allowed
}
```

Discussion

lpetroulakis

Issue fixed in [PR#15](#)

Issue L-7: Missing curve point validation

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/54>

Summary

The p256Verify precompile does not validate that the provided public key coordinates (x, y) lie on the P-256 elliptic curve before performing signature verification

Recommendation

Add curve point validation before constructing the public key, following the implementation pattern from the official Ethereum geth repository:

```
// Validate that the point is on the curve
if !elliptic.P256().IsOnCurve(x, y) {
    return nil, nil
}

// Create the public key only after validation
pubKey := &ecdsa.PublicKey{
    Curve: elliptic.P256(),
    X:     x,
    Y:     y,
}

// Verify the signature
if ecdsa.Verify(pubKey, hash, r, s) {
    return common.LeftPadBytes(common.Big1.Bytes(), 32), nil
}
return nil, nil
```

Reference the official geth implementation for secp256r1 verification:

<https://github.com/ethereum/go-ethereum/blob/b3131f00a38e1e886433cb877ad3b9701b596ed5/crypto/secp256r1/verifier.go#L28-L30>

Discussion

lpetroulakis

Issue fixed in [PR#29](#)

Issue L-8: Precompile debug print consensus risk

Source: <https://github.com/sherlock-audit/2025-07-incentiv-network/issues/60>

Summary

The WebAuthn precompile contains debug print statements that could potentially break consensus as different nodes may have different stdout configurations or I/O handling, leading to different execution behaviors.

Vulnerability Detail

The WebAuthn precompile includes `fmt.Printf()` statements for debugging purposes:

```
func (c *webAuthnVerify) Run(input []byte) ([]byte, error) {
    // ... validation logic ...

    p256Verifier := &p256Verify{}
    result, err := p256Verifier.Run(p256Input)
    if err != nil {
        fmt.Printf("P256 verification error: %v\n", err)
        return common.LeftPadBytes(common.Big0.Bytes(), 32), nil
    }
    if result == nil {
        fmt.Printf("P256 verification failed: result is nil\n")
        return common.LeftPadBytes(common.Big0.Bytes(), 32), nil
    }

    return result, nil
}
```

Code Snippet

```
// Problematic debug statements in incentivum-pow/core/vm/contracts.go:1178-1184
if err != nil {
    fmt.Printf("P256 verification error: %v\n", err) // @audit remove
    return common.LeftPadBytes(common.Big0.Bytes(), 32), nil
}
if result == nil {
    fmt.Printf("P256 verification failed: result is nil\n") // @audit remove
    return common.LeftPadBytes(common.Big0.Bytes(), 32), nil
}
```

Tool Used

Manual Review

Recommendation

Remove all debug print statements from the precompile:

```
func (c *webAuthnVerify) Run(input []byte) ([]byte, error) {
    // ... validation logic ...

    p256Verifier := &p256Verify{}
    result, err := p256Verifier.Run(p256Input)
    if err != nil {
        // Remove: fmt.Printf("P256 verification error: %v\n", err)
        return common.LeftPadBytes(common.Big0.Bytes(), 32), nil
    }
    if result == nil {
        // Remove: fmt.Printf("P256 verification failed: result is nil\n")
        return common.LeftPadBytes(common.Big0.Bytes(), 32), nil
    }

    return result, nil
}
```

Discussion

lpetroulakis

Issue fixed in [PR#9](#)

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.